

PersonaCraft AI: AI-Powered Marketing Persona Generator

Project Documentation

Date: July 1, 2026

Team ID: xxxxx

Project Description:

PersonaCraft AI harnesses Generative AI to help marketers, product managers, and founders instantly build realistic, actionable customer personas. The platform includes a Persona Profile Generator module, which builds one richly detailed persona — name, demographics, background, and a representative quote — from a short business or product description; a Pain Points & Goals Analyzer, which surfaces five core pain points and five goals that drive the persona's decisions; and a Marketing Message Generator, which delivers three tailored messaging angles (a headline, a value proposition, and a call-to-action) calibrated to the persona's mindset and the selected channel.

Utilizing Groq's API with the LLaMA 3.3-70B Versatile model, PersonaCraft AI processes user inputs to deliver fast, high-quality persona content. Built with Flask and deployed publicly via Ngrok, the platform ensures a seamless and user-friendly experience. With secure API key management via .env and responsible input handling, PersonaCraft AI empowers marketing teams to understand their audience and craft campaigns with confidence.

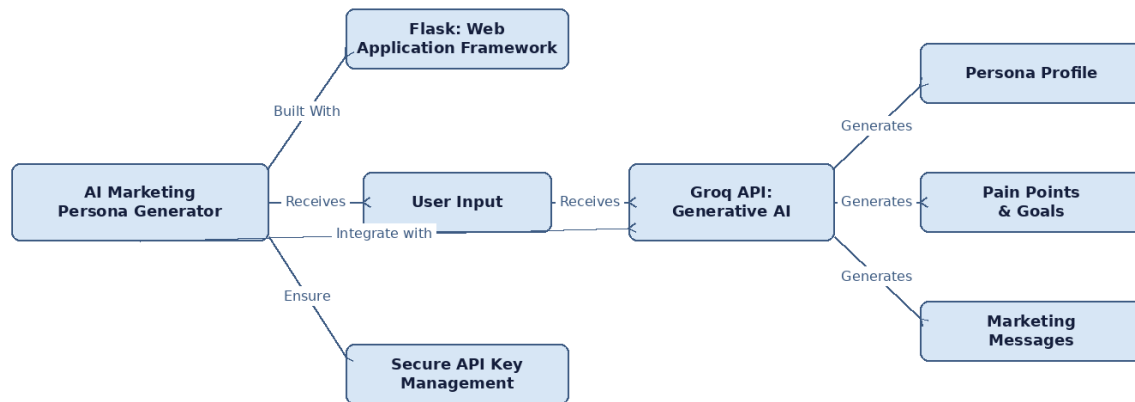
Scenarios

Scenario 1: A user describes a new eco-friendly skincare line and selects Instagram with a Casual tone. The system generates a persona named "Meera, 27, sustainability-conscious urban professional," five pain points and goals around clean beauty and budget, and three emoji-rich, conversational marketing messages — all in seconds.

Scenario 2: A user promoting a B2B project-management SaaS selects LinkedIn with a Professional tone. PersonaCraft AI generates a persona named "Ravi, 38, IT Director at a mid-size enterprise," enterprise-grade pain points around team visibility and budget approval, and thought-leadership-style messaging designed to drive demo sign-ups.

Scenario 3: A user running a weekend flash sale for a D2C sneaker brand selects Instagram with a Promotional tone. The AI generates a deal-seeking Gen Z shopper persona, urgency-driven pain points and goals, and punchy, power-word messaging like "Don't miss your pair — sale ends Sunday!" to maximize conversion.

Technical Architecture:



Description: PersonaCraft AI utilizes a modular three-tier architecture to deliver rapid, AI-driven persona generation. The frontend provides a responsive user interface built with HTML and CSS while the Flask-based backend orchestrates data validation and prompt construction. It interfaces with the Groq API using the LLaMA 3.3-70B model to generate a tailored persona profile, pain points and goals, and marketing messages, with secure deployment managed locally via Flask and exposed publicly through Ngrok.

(Note: If you are using a database in your project, definitely you need to add an ER Diagram along with a description.)

Pre-requisites:

- Flask Framework Knowledge: [Flask Documentation](#)
- Groq API Familiarity: [Groq API Documentation](#)
- HTML, CSS, and JavaScript Skills: [W3Schools HTML/CSS/JavaScript Tutorials](#)
- Python Programming Proficiency: [Python Documentation](#)
- Version Control with Git: [Git Documentation](#)
- Development Environment Setup: [Flask Installation Guide](#)
- Ngrok for Public Deployment: [Ngrok Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for persona content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Persona Profile Generator, Pain Points & Goals Analyzer, and Marketing Message Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our customer persona generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating persona profiles, pain points and goals, and marketing messages across different channels and tones.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Groq Account:** Visit console.groq.com and sign up for a free account using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.
- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., `personacraft-key`) so it is easy to identify later.
- **Step 4 — API Key:** Give a "Display Name" ("Expiration" is optional) and click "Submit". You will get your API key — copy it immediately and store it in a safe place, as you will not be able to view it again after closing the dialog.
- **Step 5 — Add the Key to Your Project:** Create a `.env` file in the root of your project directory (if it does not already exist) and paste the key as shown below:
- `GROQ_API_KEY=your_copied_api_key_here`
- **Step 6 — Verify the Key is Loaded:** The `app.py` file loads this key automatically at startup using `python-dotenv`:
- ```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

**Important — Never Commit Your API Key:** Add `.env` to your `.gitignore` file to prevent accidentally pushing the key to a public repository:

`.env`

### Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the PersonaCraft AI application, focusing on the types of content it will generate (persona profile, pain points and goals, and marketing messages across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to audience-research and copywriting tasks.

- **Select the Optimal Model:** Choose llama-3.3-70b-versatile for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max\_tokens of 2048 for rich, well-rounded personas.

### Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application — entering business/product information, selecting a channel (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE\_DESCRIPTIONS and CHANNEL\_TIPS dictionaries, calls the Groq API, parses the JSON response via extract\_json(), and returns structured results to the frontend.

### Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:
- 
- D:\projects>python -m venv myenv
- D:\projects>"d:\projects\venv\Scripts\activate"
- (myenv) D:\projects> pip install -r requirements.txt
- 
- **Install Flask:** Use pip to install Flask:
- 
- (myenv) D:\projects> pip install Flask==3.0.3
- 
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:
- 
- (myenv) D:\projects> pip install groq==0.11.0
- 
- **Install python-dotenv:** Install dotenv support for secure API key management:
- 
- (myenv) D:\projects> pip install python-dotenv==1.0.1
- 
- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
- 
- GROQ\_API\_KEY=your\_groq\_api\_key\_here

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run\_public.py).
- 
- static/
- css/
- js/
- templates/
- .env
- .gitignore
- app.py
- requirements.txt
- run\_public.py

## Milestone 2: Core Functionalities Development

### Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features — Persona Profile, Pain Points & Goals, and Marketing Messages — as a unified AI response driven by a single structured prompt.

- **Persona Profile Generator:** Generates one richly detailed persona including name, age, occupation, background, and a representative quote. For Instagram/Casual tone, the persona reads relatable and conversational. For LinkedIn/Professional tone, the persona reads like a credible business decision-maker.
- **Pain Points & Goals Analyzer:** Produces five pain points and five goals per request — a balanced mix of emotional and practical motivations relevant to the business or campaign.
- **Marketing Message Generator:** Returns three short, punchy messaging angles (headline, value proposition, and call-to-action) calibrated to the selected channel and tone.

### Activity 2.2: Implement the Flask Backend

#### Define Routes in Flask:

- Set up routes in `app.py` for the home page and content generation endpoint.

#### Process User Input:

- Create a form in `index.html` for users to submit their business/product information, channel, and tone.
- Use Flask's `request.get_json()` to retrieve JSON data submitted from the frontend via AJAX.
- Validate that `business_info` is non-empty and within the 2000-character limit before processing.

#### Integrate API Calls:

- Within the `/generate` route handler, call the `build_prompt()` function to construct a structured prompt combining tone descriptions and channel-specific tips.
- Submit the prompt to the Groq API using `client.chat.completions.create()` with the `llama-3.3-70b-versatile` model.
- Parse and validate the JSON response using `extract_json()`, which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

## Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the PersonaCraft AI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

### Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- `/` — Home page route that renders the main index.html interface
- 
- `@app.route("/")`
- `def index():`
- `return render_template("index.html")`
- 
- `/generate` — POST endpoint that accepts JSON input, invokes the AI, and returns structured content
- 
- `@app.route("/generate", methods=["POST"])`
- `def generate():`
- `data = request.get_json()`
- `business_info = data.get("business_info", "").strip()`
- `channel = data.get("channel", "instagram").lower()`
- `tone = data.get("tone", "professional").lower()`
- 
- `if not business_info:`
- `return jsonify({"error": "Business information is required."}), 400`
- `if len(business_info) > 2000:`
- `return jsonify({"error": "Input too long. Please keep it under 2000 characters."}), 400`
- 
- `try:`
- `prompt = build_prompt(business_info, channel, tone)`
- 
- `chat_completion = client.chat.completions.create(`
- `messages=[`
- `{`
- `"role": "system",`
- `"content": "You are a customer research and marketing expert. Always respond with valid JSON only, no markdown formatting."`

```

• },
• {
• "role": "user",
• "content": prompt
• }
•],
• model="llama-3.3-70b-versatile",
• temperature=0.8,
• max_tokens=2048,
•)
•
• response_text = chat_completion.choices[0].message.content
• result = extract_json(response_text)
•
• # Validate structure
• if not all(k in result for k in ["persona", "pain_points_goals", "messages"]):
• raise ValueError("Invalid response structure from AI")
•
• return jsonify({
• "success": True,
• "persona": result["persona"],
• "pain_points_goals": result["pain_points_goals"],
• "messages": result["messages"],
• "channel": channel,
• "tone": tone
• })
• except ValueError as e:
• import traceback
• traceback.print_exc()
• return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500
• except Exception as e:
• import traceback
• traceback.print_exc()
• return jsonify({"error": f"Generation failed: {str(e)}"}), 500

```

Set Up Route Handlers for Each Feature:

- For the `/generate` route, implement logic that reads `business_info`, `channel`, and `tone` from the incoming JSON body using `request.get_json()`.
- Apply input validation: return a 400 error if `business_info` is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the `/generate` endpoint, which processes data and returns a JSON response.

### Define Prompt Engineering Helpers:

- **TONE\_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed persona-writing instructions embedded in the prompt.
- 
- `TONE_DESCRIPTIONS = {`
- `"professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",`
- `"casual": "friendly, conversational, and relatable. Use informal language and a warm tone that feels personal.",`
- `"promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."`
- `}`
- 
- **CHANNEL\_TIPS** dictionary: maps instagram and linkedin to channel-specific persona and messaging guidelines.
- 
- `CHANNEL_TIPS = {`
- `"instagram": "optimized for Instagram audiences — visual, emotionally driven, and community-oriented.",`
- `"linkedin": "optimized for LinkedIn audiences — professional insights, value-driven, thought leadership style."`
- `}`
- 
- **build\_prompt()**: assembles a structured prompt string from the business info, selected channel tip, and tone description, instructing the model to return strictly valid JSON.
- 
- `def build_prompt(business_info: str, channel: str, tone: str) -> str:`
- `tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")`
- `channel_tip = CHANNEL_TIPS.get(channel, "general marketing")`
- 
- `return f"""You are an expert customer researcher and marketing strategist.`

### Integrate the Groq AI Responses in Each Function:

- Call `client.chat.completions.create()` with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract\_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object — ensuring reliable parsing across model response variations.
- 
- `def extract_json(text: str) -> dict:`
- `"""Extract JSON from LLM response, handling markdown code blocks."""`
- `# Try direct parse first`
- `try:`
- `return json.loads(text)`
- `except json.JSONDecodeError:`
- `pass`
- 
- `# Try extracting from markdown code block`
- `match = re.search(r'```(?:json)?\s*([^\s\S]*)```', text)`
- `if match:`
- `try:`
- `return json.loads(match.group(1).strip())`
- `except json.JSONDecodeError:`
- `pass`
- 
- `match = re.search(r'\{[^\s\S]*\}', text)`
- `if match:`
- `try:`
- `return json.loads(match.group(0))`
- `except json.JSONDecodeError:`
- `pass`
- 
- `raise ValueError("Could not extract valid JSON from response")`
- 
- Validate that the parsed result contains all three required keys — `persona`, `pain_points_goals`, and `messages` — before returning it to the frontend.
- **Home route:** `/` → renders `index.html` `/generate` (POST) → accepts JSON, returns `{success, persona, pain_points_goals, messages, channel, tone}`

## Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for PersonaCraft AI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display the AI-generated persona profile, pain points and goals, and marketing messages based on user interactions.

### Activity 4.1: Designing and Developing the User Interface

#### Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the PersonaCraft AI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for channel indicators and result section headings, and Google Fonts (Inter) for clean typography.

#### Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (persona profile on the left, pain points/goals and messages on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

#### Create Separate UI Components for Each Output Type:

- **Persona Card:** a single glass card showing the persona's name, age, occupation, background, and quote, with a one-click copy-to-clipboard button.
- **Pain Points & Goals Chips:** rendered as pill-shaped chip elements inside a glass card, grouped visually by pain points versus goals.
- **Marketing Message List:** displayed as a clean list inside a separate glass card with a bullhorn icon header, labeling each message as Headline, Value Proposition, or CTA.

### Activity 4.2: Creating Dynamic Content Rendering with JavaScript

#### Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

#### Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for the persona, pain points/goals, and messages into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollToView().

#### Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

## Milestone 5: Deployment

In Milestone 5, the focus is on deploying the PersonaCraft AI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

### Activity 5.1: Preparing the Application for Local Deployment

#### Set Up a Virtual Environment:

- Open a new terminal in your code editor and select Command Prompt (or your preferred shell).
- Create and activate a virtual environment, then install all required dependencies from requirements.txt.
- 
- `D:\projects>python -m venv myenv`
- `D:\projects>"d:\projects\venv\Scripts\activate"`
- `(myenv) D:\projects>pip install -r requirements.txt`

#### Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:
- `GROQ_API_KEY=your_groq_api_key_here`

### Activity 5.2: Local Testing and Verification

#### Start the Flask Development Server:

- Run the application locally using:
- 
- `(myenv) D:\projects>python app.py`
- 
- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.
- Interact with the persona generator — test all three tones (Professional, Casual, Promotional) and both channels (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

### Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

#### Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:
- 
- `D:\projects>pip install pyngrok`

- Download and install the Ngrok client from [ngrok.com/download](https://ngrok.com/download).

### Configure Your Ngrok Authtoken:

- Sign up at [ngrok.com](https://ngrok.com) and copy your authtoken from the Ngrok dashboard.
- The `run_public.py` script sets the authtoken automatically using:
- 
- `D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")`
- Replace the TOKEN value in `run_public.py` with your own Ngrok authtoken before running.

### Run the Public Deployment Script:

- Instead of running `app.py` directly, launch `run_public.py` to start both the Ngrok tunnel and the Flask server:
- 
- `D:\projects>python run_public.py`
- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:
- 
- =====
- YOUR PUBLIC WEBSITE URL IS LIVE!
- URL: `https://xxxx-xx-xx-xxx-xx.ngrok-free.app`
- =====
- Share this URL with anyone who can access your PersonaCraft AI app directly from their browser without any installation.

### Important Notes:

- **debug=False** is required in `run_public.py` to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart `run_public.py` (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart `run_public.py` to get a new URL when this happens.

## Exploring the Website's Web Pages:

### Home / Generator Page:

**Description:** The single-page interface of PersonaCraft AI serves as both the landing page and the generation tool. It features a dark glassmorphism design with animated background blobs, a header displaying the PersonaCraft AI brand (target-audience icon + logo), and a tagline. The main section contains the input form, channel toggle, tone selector, and the AI-generated results panel all on one seamless page.

### Input Section:

**Description:** A glass-styled card containing the main generation form. Users enter a description of their business, product, or campaign in a textarea (up to 2000 characters), select their target channel via a toggle button group (Instagram or LinkedIn), choose their tone of voice (Professional, Casual, or Promotional), and click the Generate Persona button. While the AI processes the request, the button transitions to an animated loading spinner.

### Results Section:

**Description:** Revealed dynamically after generation, this section displays the AI output in a two-column grid. The left column shows the persona card with name, demographics, background, and quote plus a copy-to-clipboard button. The right column contains two glass cards: one with pain point and goal chips, and one with the three marketing messages in a labeled list format. The viewport automatically smooth-scrolls to this section upon content generation.

## Conclusion:

PersonaCraft AI is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines customer research for marketing teams. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate a tailored persona profile, pain points and goals, and marketing messages for Instagram and LinkedIn across professional, casual, and promotional tones. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost productivity for marketers and product teams, with potential for future scalability. Looking ahead, PersonaCraft AI offers significant opportunities for expansion, such as generating multiple personas per business, adding competitor and market-context analysis, and implementing user authentication to save and revisit past personas. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive audience-research assistant for teams of all sizes.